

Salesforce Apex Assignment

Question

When a maintenance request is created, automatically assign the request to the vendor with the least assigned workload.

Solution Approach

For this requirement, I implemented the solution using:

1. Apex Trigger
2. Trigger Handler Class
3. Test Class

The trigger executes before inserting Maintenance Request records and calls the handler class.

The handler class performs the following operations:

- Fetches all available vendors
- Calculates current vendor workload using Aggregate SOQL
- Identifies the vendor with the least workload
- Assigns the vendor dynamically to new maintenance requests
- Updates workload count dynamically for bulk records

This solution is bulkified and follows Salesforce best practices.

Features Implemented

- Automatic Vendor Assignment
- Least Workload Distribution
- Bulk Record Handling
- Trigger Handler Pattern
- AggregateResult Usage
- No SOQL inside loops
- No DML inside loops
- Test Class with Assertions

Files Included

1. MaintenanceRequestTrigger.trigger
2. MaintenanceRequestTriggerHandler.cls
3. MaintenanceRequestTriggerTest.cls

Trigger Handler Class

```
public class MaintenanceRequestTriggerHandler {

    public static void assignVendor(List<Maintenance_Request__c> requests) {

        List<Vendor__c> vendors = [SELECT Id FROM Vendor__c];

        if (vendors.isEmpty()) return;

        Map<Id, Integer> workloadMap = new Map<Id, Integer>();

        for (Vendor__c v : vendors) {
            workloadMap.put(v.Id, 0);
        }

        for (AggregateResult ar : [
            SELECT Vendor__c vId, COUNT(Id) total
            FROM Maintenance_Request__c
                WHERE Status__c NOT IN ('Completed', 'Cancelled')
            AND Vendor__c != NULL
            GROUP BY Vendor__c
        ]) {
            workloadMap.put((Id) ar.get('vId'), (Integer) ar.get('total'));
        }

        for (Maintenance_Request__c req : requests) {

            Id selectedVendorId;
            Integer minCount;
            Boolean isFirst = true;

            for (Id vId : workloadMap.keySet()) {
                Integer current = workloadMap.get(vId);

                if (isFirst || current < minCount) {
                    minCount = current;
                    selectedVendorId = vId;
                    isFirst = false;
                }
            }

            req.Vendor__c = selectedVendorId;
        }
    }
}
```

```

        workloadMap.put(selectedVendorId, minCount + 1);
    }
}

```

Trigger

```

trigger MaintenanceRequestTrigger on Maintenance_Request__c (before insert)
{
    if (Trigger.isBefore && Trigger.isInsert)
    {
        MaintenanceRequestTriggerHandler.assignVendor(Trigger.new);
    }
}

```

Test Class

```

@Test
public class MaintenanceRequestTriggerTest {

    @testSetup
    static void setupData() {

        List<Vendor__c> vendors = new List<Vendor__c>();

        for (Integer i = 1; i <= 3; i++) {
            vendors.add(new Vendor__c(Name = 'Vendor ' + i));
        }
        insert vendors;

        List<Maintenance_Request__c> existingRequests = new
        List<Maintenance_Request__c>();

        existingRequests.add(new Maintenance_Request__c(
            Status__c = 'In Progress',
            Vendor__c = vendors[0].Id

```

```

));

existingRequests.add(new Maintenance_Request__c(
    Status__c = 'In Progress',
    Vendor__c = vendors[0].Id
));

existingRequests.add(new Maintenance_Request__c(
    Status__c = 'In Progress',
    Vendor__c = vendors[1].Id
));

insert existingRequests;
}

@IsTest
static void testVendorAssignmentLogic() {

    List<Maintenance_Request__c> newRequests = new List<Maintenance_Request__c>();

    for (Integer i = 0; i < 5; i++) {
        newRequests.add(new Maintenance_Request__c(
            Status__c = 'New'
        ));
    }

    Test.startTest();
    insert newRequests;
    Test.stopTest();

    List<Maintenance_Request__c> insertedRequests = [
        SELECT Id, Vendor__c
        FROM Maintenance_Request__c
        WHERE Id IN :newRequests
    ];

    System.assertEquals(5, insertedRequests.size());

    for (Maintenance_Request__c req : insertedRequests) {
        System.assertNotEquals(null, req.Vendor__c);
    }
}
}

```

Outcome

The solution successfully assigns maintenance requests to vendors based on the least assigned workload while supporting bulk processing and maintaining reusable Apex code architecture.